Listing 3.11. Thread-safe lazy initialization using a mutex

Listing 3.12. Thread-safe lazy initialization of a class member using std::call_once

Listing 3.13. Protecting a data structure with a boost::shared_mutex

Chapter 4. Synchronizing concurrent operations

Listing 4.1. Waiting for data to process with a std::condition_variable

Listing 4.2. std::queue interface

Listing 4.3. The interface of your threadsafe_queue

Listing 4.4. Extracting push() and wait_and_pop() from listing 4.1

Listing 4.5. Full class definition for a thread-safe queue using condition variables

Listing 4.6. Using std::future to get the return value of an asynchronous task

Listing 4.7. Passing arguments to a function with std::async

Listing 4.8. Partial class definition for a specialization of std::packaged_task< >

Listing 4.9. Running code on a GUI thread using std::packaged_task

Listing 4.10. Handling multiple connections from a single thread using promises

Listing 4.11. Waiting for a condition variable with a timeout

Listing 4.12. A sequential implementation of Quicksort

Listing 4.13. Parallel Quicksort using futures

Listing 4.14. A sample implementation of spawn_task

Listing 4.15. A simple implementation of an ATM logic class

Listing 4.16. The getting_pin state function for the simple ATM implementation

Chapter 5. The C++ memory model and operations on atomic types